

Phase 1 Microservices Project Documentation

Project Title

Service-Oriented User Management System

Team Members

- Moataz Kamal — ID: 232400656
 - Yousef Sherif — ID: 232400207
-

1. Project Overview

This project demonstrates the design and implementation of a **service-oriented microservices architecture** using Node.js, Express, MongoDB, and React.

The goal of the system is to build a scalable backend composed of multiple independent services and a frontend dashboard to manage users.

The system includes:

- Authentication Service
- User Service
- Admin Service
- React Frontend Dashboard
- Postman API testing collection

The architecture separates responsibilities between services, allowing better scalability, maintainability, and security.

2. Technologies Used

Backend

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT Authentication
- bcryptjs

Frontend

- React
- Vite
- Axios
- React Router

Tools

- Postman
 - GitHub
 - MongoDB Atlas
-

3. System Architecture

The project follows a **microservices architecture** where each service has its own responsibilities.

Services communicate using REST APIs.

System structure:

Auth Service (Port 5001) Handles: - User registration - User login - JWT token generation

User Service (Port 5002) Handles: - Logged-in user information - Welcome message

Admin Service (Port 5003) Handles: - User management - Create users - Update users - Delete users - View users

Frontend (React) Provides: - Login interface - Register interface - Admin dashboard - User management UI

4. Backend Services

4.1 Auth Service

Purpose

The authentication service is responsible for handling user login and registration and generating JWT tokens.

Main Endpoints

POST /api/auth/register Registers a new user.

POST /api/auth/login Authenticates a user and returns a JWT token.

Security

- Passwords are hashed using bcrypt.
- JWT tokens are used for authentication.

Example login response:

```
{ "success": true, "data": { "token": "JWT_TOKEN", "user": { "id": "...", "email": "...", "role": "admin" } } }
```

4.2 User Service

Purpose

Handles operations related to the logged-in user.

Endpoints

GET /api/users/me Returns the current authenticated user's information.

GET /api/users/welcome Returns a personalized welcome message.

Example response:

```
{ "message": "Welcome Moataz" }
```

4.3 Admin Service

Purpose

Allows administrators to manage system users.

Role Protection

All admin routes require:

1. Valid JWT token
2. User role must be "admin"

If a normal user accesses admin routes, the system returns:

403 Forbidden

Endpoints

GET /api/admin/users Returns all users.

GET /api/admin/users/:id Returns a specific user.

POST /api/admin/users Creates a new user.

PUT /api/admin/users/:id Updates a user.

DELETE /api/admin/users/:id Deletes a user.

5. Database Design

The system uses MongoDB with the following User schema:

User

Fields:

name email password role createdAt updatedAt

Role values:

admin user

Passwords are stored as hashed values.

6. Authentication Flow

1. User registers
2. Password is hashed
3. User logs in
4. JWT token is generated
5. Token is used in protected routes

Header format:

Authorization: Bearer TOKEN

Middleware verifies:

JWT validity User role

7. Frontend Dashboard

The frontend application is built using **React + Vite**.

Pages

Home Login Register Admin Dashboard

Features

User authentication Token storage in localStorage Protected routes Admin dashboard User management

Dashboard Capabilities

Admin can:

View all users Create users Update users Delete users

Normal users:

Cannot access the admin dashboard.

8. Postman API Testing

Postman was used to test all backend APIs.

The collection includes the following requests:

Auth Service

Register Login

User Service

Get Profile Welcome Message

Admin Service

Get All Users Get User By ID Create User Update User Delete User

The Postman collection was exported as:

Phase1_Microservices_API.postman_collection.json

This file can be imported directly into Postman.

9. Example API Requests

Login

POST

http://localhost:5001/api/auth/login

Body

```
{ "email": "moataz@gmail.com", "password": "1111" }
```

Get All Users

GET

http://localhost:5003/api/admin/users

Header

Authorization: Bearer TOKEN

Create User

POST

http://localhost:5003/api/admin/users

Body

```
{ "name": "Ali", "email": "ali@gmail.com", "password": "123456", "role": "user" }
```

10. Project Folder Structure

phase1-project

admin-service src controllers middleware models routes services

user-service src

auth-service src

frontend src components pages services

11. Security Implementation

Security features implemented:

Password hashing using bcrypt JWT authentication Role-based authorization Protected API routes Admin-only endpoints

12. Testing

The system was tested using:

Postman React frontend Manual API testing

All CRUD operations were verified.

13. Conclusion

This project successfully demonstrates a service-oriented architecture with multiple independent services and a React frontend interface.

The system supports:

Authentication Authorization User management Role-based access control

The microservices design ensures the system is modular, scalable, and maintainable.

Future improvements may include:

Docker containerization API gateway Logging services Monitoring Deployment to cloud infrastructure.